LECTURE 11

CEN 348-Artificial Intelligence Systems
Beyond classical search

Ezgi Zorarpacı

Local search algorithms and optimization problems

- Hill climbing search
- Simulated annealing
- Local beam search
- Genetic algorithms

Local Search

- The *uninformed and informed search algorithms* that we have seen are designed to explore search spaces systematically.
  - They keep one or more paths in memory and by record which alternatives have been explored at each point along the path.
  - When a goal is found, *the path to that goal also constitutes a solution to the problem*.
  - In many problems, however, the path to the goal is irrelevant.

- If ***the path to the goal does not matter***, we might consider a different class of algorithms that do not worry about paths at all.

  ➔ **local search algorithms**

Local Search

- **Local search algorithms** operate using a *single current node* and generally move only to neighbors of that node.
- **Local search algorithms** ease up on ***completeness and optimality*** in the interest of ***improving time and space complexity***?
- Although **local search algorithms** are not ***systematic***, they have ***two key advantages***:
  1. They use ***very little memory*** (usually a constant amount), and
  2. They can often find ***reasonable solutions*** in large or infinite (continuous) state spaces.

- In addition to finding goals, **local search algorithms** are useful for solving pure **optimization problems**, in which the aim is ***to find the best state*** according to an ***objective function***.
  - In optimization problems, ***the path to goal is irrelevant*** and ***the goal state itself is the solution.***
  - In some optimization problems, the ***goal is not known*** and ***the aim is to find the best state***.

Hill Climbing Search
(Steepest Ascent/Descent)

- At each iteration, the **hill-climbing search algorithm** moves to the ***best successor*** of the ***current node*** according to an ***objective function.***
  - Best successor is the successor with best value (highest or lowest) according to an objective function.
  - If no successors have better value than the current value, it returns.
  - It moves in direction of uphill (hill climbing).
  - It terminates when it reaches a "peak" where no neighbor has a higher value.
- The algorithm does not maintain a search tree, so the data structure for the current node need only record the state and the value of the objective function.
- **Hill climbing** does not look ahead beyond the immediate neighbors of the current state.
- **Hill climbing** is sometimes called ***greedy local search*** because it grabs a good neighbor state without thinking ahead about where to go next.
  - Greedy algorithms often perform quite well and
  - Hill climbing often makes rapid progress toward a solution.

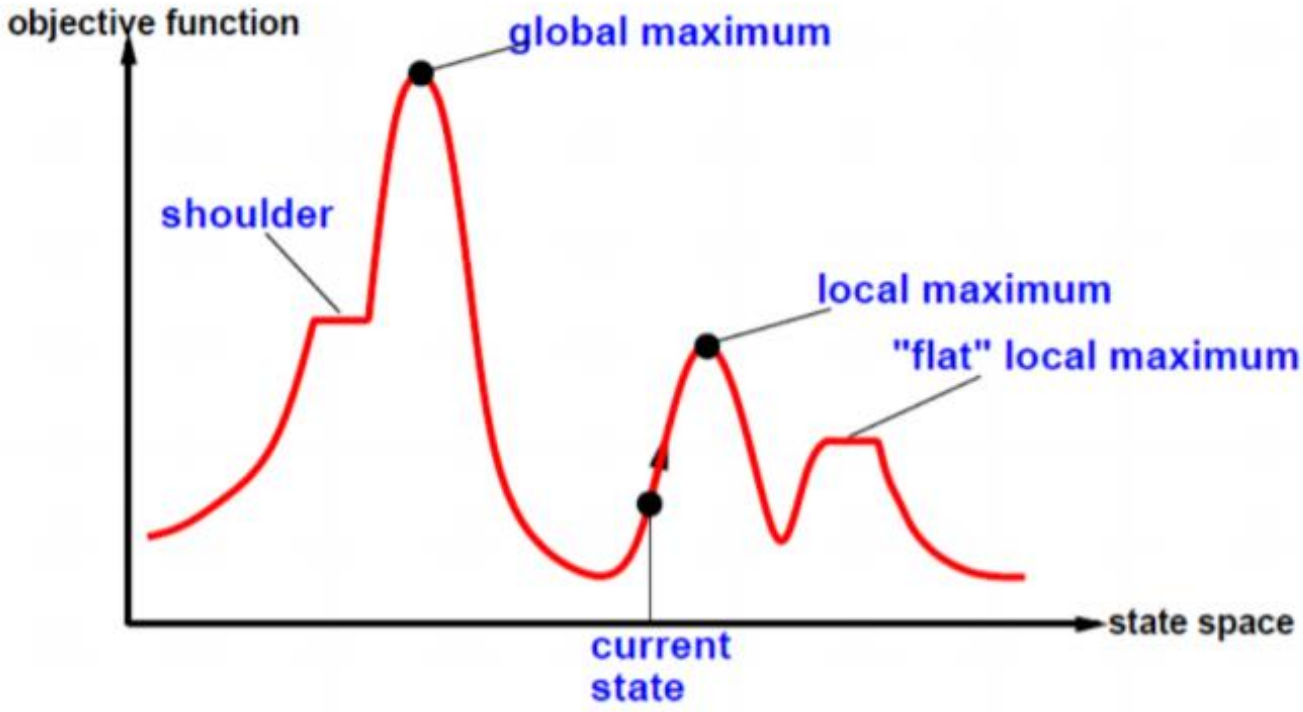
Hill Climbing Search
(Steepest Ascent/Descent)

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current ← MAKE-NODE(*problem*.INITIAL-STATE)
loop do
neighbor ← a highest-valued successor of *current*
if neighbor.VALUE ≤ current.VALUE **then return** *current*.STATE
current ← *neighbor*

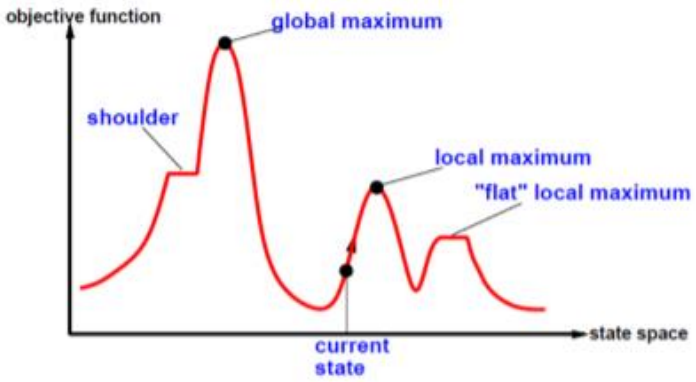
Hill Climbing Search

- To understand local search, it is useful to consider the ***state-space landscape.***
- The aim is to find the highest peak - **a global maximum**.
- Hill-climbing search modifies the current state to try to improve it.



Hill Climbing Search

- A **local maximum** is a peak that is higher than each of its neighboring states but lower than the **global maximum**.
  - Hill-climbing algorithms that reach the vicinity of a local maximum will be drawn upward toward the peak but will then be stuck with nowhere else to go.
- A **plateau** is a **flat area** of the state-space landscape. It can be a **flat local maximum**, from which no uphill exit exists, or a **shoulder**, from which progress is possible.
  - A hill-climbing search might get lost on the plateau.
  - **Random sideways moves** can escape from ***shoulders*** but they loop forever on ***flat maxima.***



Hill Climbing Example

n-queens

- Put n queens on an n × n board with no two queens on the same row, column, or diagonal
- Move a queen to reduce number of conflicts.

➔ **Objective function: number of conflicts (no conflicts is global minimum)**

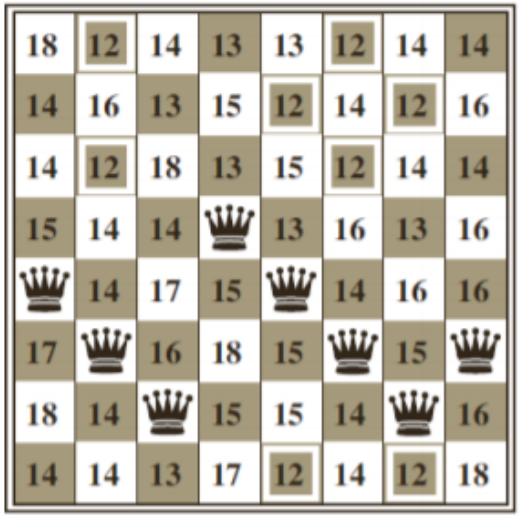
h = 5 h = 2 h = 0

- The successors of a state are all possible states generated by moving a single queen to another square in the same column (so each state has n*(n-1) successors).
- The heuristic cost function h is the number of pairs of queens that are attacking each other, either directly or indirectly.
- The global minimum of this function is zero, which occurs only at perfect solutions.

Hill Climbing Example

n-queens

- Hill Climbing may NOT reach to a goal state for n-queens problem.



➔ five steps to reach this state

- An 8-queens state with heuristic cost estimate h=17
- The value of h for each possible successor obtained by moving a queen within its column.
- The best moves are marked with value 12.

- A local minimum in the 8-queens state space; the state has h=1
- but every successor has a higher cost.
- Hill Climbing will stuck here

8-queens example with Hill Climbing

- https://www.youtube.com/watch?v=vEpPMIiTSDI

Hill Climbing Example

n-queens

- Starting from a randomly generated 8-queens state, **steepest-ascent hill climbing** gets stuck 86% of the time, solving only 14% of problem instances.

- The Hill Climbing algorithm halts if it reaches a **plateau.**
  - One possible solution is to allow **sideways move** in the hope that the **plateau** is really a **shoulder**.
  - If we always allow sideways moves when there are no uphill moves, an infinite loop will occur whenever the algorithm reaches a flat local maximum that is not a shoulder.
  - One common solution is to put a limit on the number of consecutive sideways moves allowed.
  - For example, we could allow up to 100 consecutive sideways moves in the 8-queens problem. This raises the percentage of problem instances solved by hill climbing from 14% to 94%.

8 puzzle example with Hill Climbing

- https://www.youtube.com/watch?v=uJA0i90uCGE

Hill Climbing Example

8-puzzle

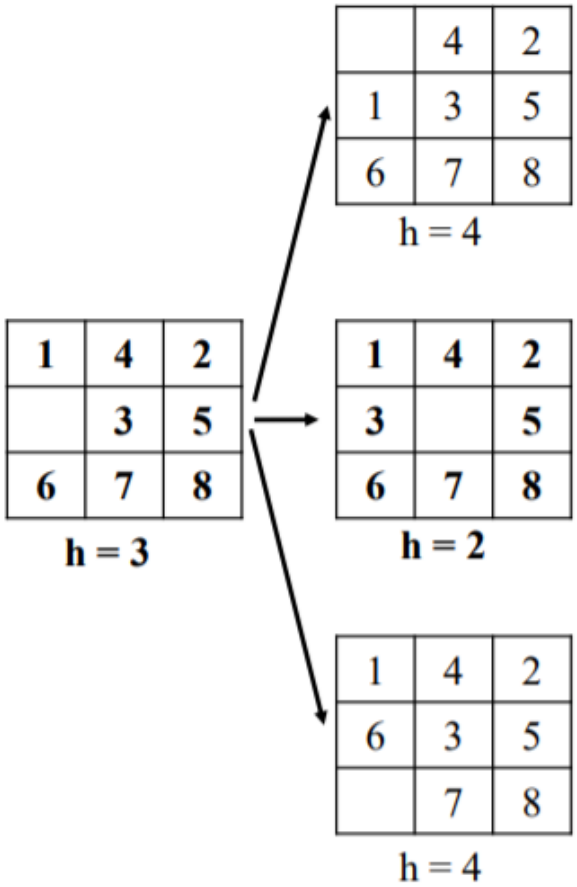
Heuristic function is Manhattan Distance

1	4	2
	3	5
6	7	8

h = 3

Hill Climbing Example

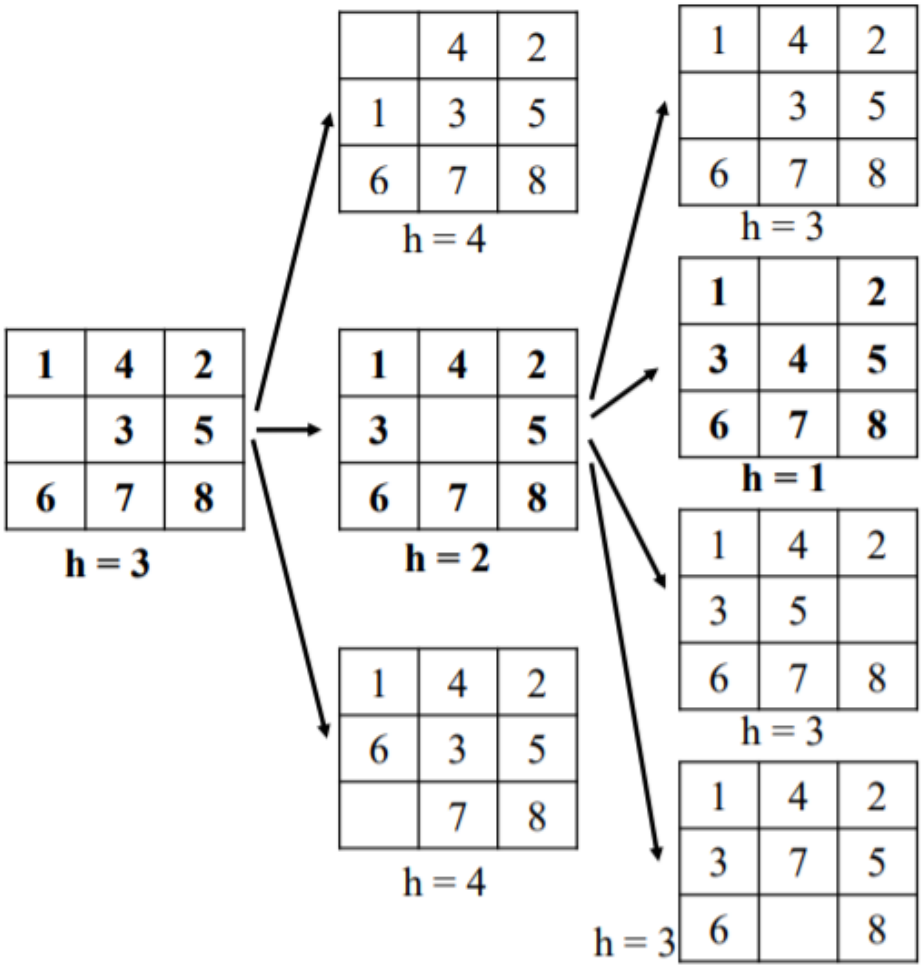
8-puzzle



Heuristic function is Manhattan Distance

Hill Climbing Example

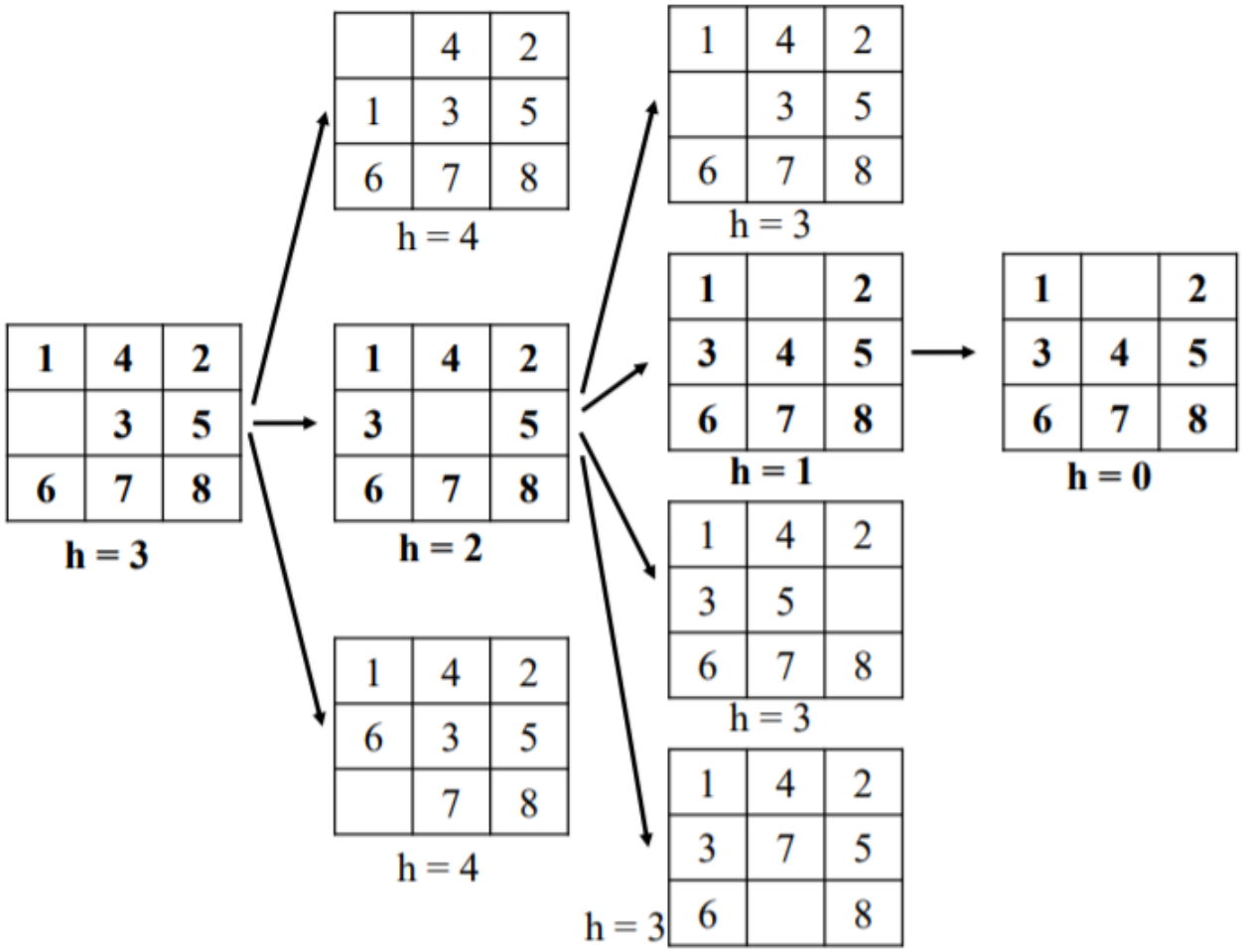
8-puzzle



Heuristic function is Manhattan Distance

Hill Climbing Example

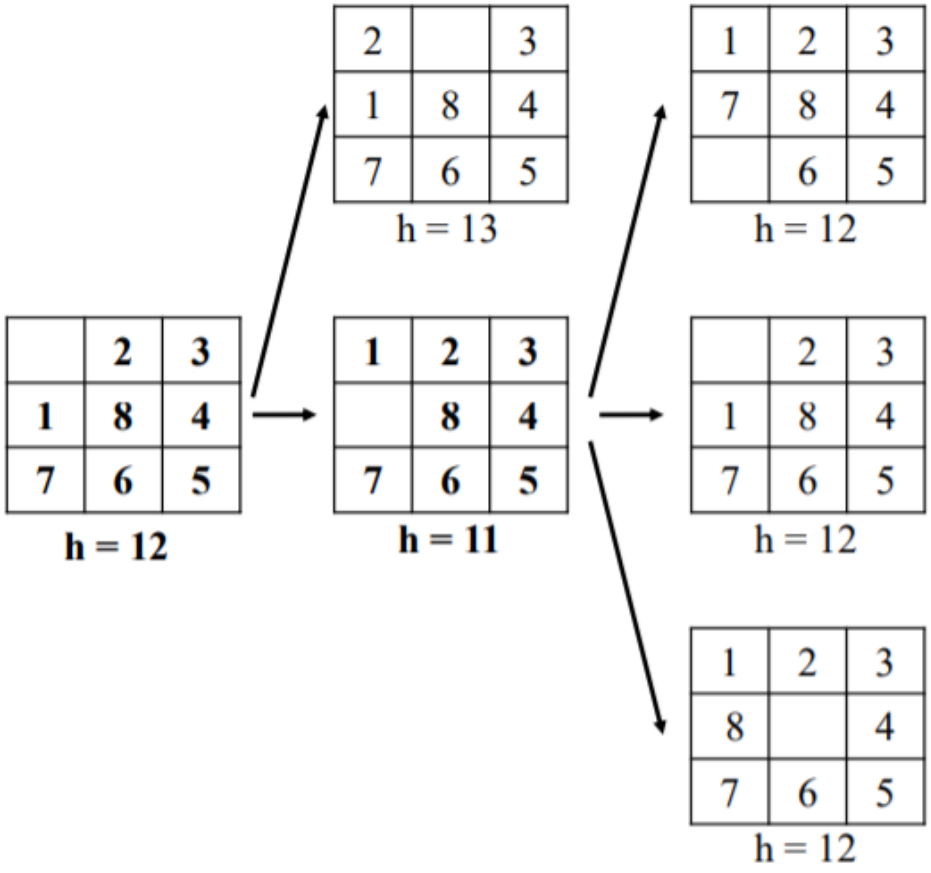
8-puzzle: a solution case



Heuristic function is Manhattan Distance

Hill Climbing Example

8-puzzle: stuck at local maximum



Heuristic function is Manhattan Distance

We are stuck with a local maximum.

Hill Climbing

- Hill Climbing is **NOT complete**.
- Hill Climbing is **NOT optimal**.

- **Why use local search?**
  - Low memory requirements – usually constant
  - Effective – Can often find good solutions in extremely large state spaces
  - Randomized variants of hill climbing can solve many of the drawbacks in practice.

- Many variants of hill climbing have been invented.

Stochastic Hill Climbing

- **Stochastic hill climbing** chooses at random from among the uphill moves;
- The **probability of selection** can vary with the ***steepness of the uphill move***.
- **Stochastic hill climbing** usually converges more slowly than steepest ascent, but in some state landscapes, it finds better solutions.

- **Stochastic hill climbing** is NOT complete, but it may be less likely to get stuck.

First-Choice Hill Climbing

- **First-choice hill climbing** implements ***stochastic hill climbing*** by generating successors randomly until one is generated that is better than the current state.
- This is a good strategy when a state has many of successors.

- **First-choice hill climbing** is also NOT complete,

Random-Restart Hill Climbing

- **Random-Restart Hill Climbing** conducts a ***series of hill-climbing searches*** from ***randomly generated initial states***, until a ***goal is found***.
- Random-Restart Hill Climbing is **complete** if ***infinite (or sufficiently many tries) are allowed***.
- If each hill-climbing search has a ***probability p*** of success, then the ***expected number of restarts required*** is **1/p**.
- For 8-queens instances with no sideways moves allowed, $p \approx 0.14$, so we need roughly 7 iterations to find a goal (6 failures and 1 success).
  - For 8-queens, then, random-restart hill climbing is very effective indeed.
  - Even for three million queens, the approach can find solutions in under a minute.
- The ***success of hill climbing*** depends very much on the ***shape of the state-space landscape***:
  - If there are few local maxima and plateau, ***random-restart hill climbing*** will find a good solution very quickly.
  - On the other hand, many real problems have ***many local maxima*** to get stuck on.
  - NP-hard problems typically have an ***exponential number of local maxima*** to get stuck on.

Simulated Annealing

- A **hill-climbing algorithm** that never makes "downhill" moves toward states with lower value (or higher cost) is guaranteed to be **incomplete**, because ***it can get stuck on a local maximum***.
  - In contrast, a **purely random walk**—that is, moving to a successor chosen uniformly at random from the set of successors—is **complete but extremely inefficient**.
  - Therefore, it seems reasonable to ***combine hill climbing with a random walk in some way that yields both efficiency and completeness***.
- **Idea:** ***escape local maxima by allowing some "bad" moves but gradually decrease their size and frequency.***
- The **simulated annealing algorithm**, a version of ***stochastic hill climbing*** where some downhill moves are allowed.
- **Annealing:** the process of gradually cooling metal to allow it to form stronger crystalline structures
- **Simulated annealing algorithm:** gradually "cool" search algorithm from Random Walk to First-Choice Hill Climbing

Motivation

- Annealing in metals
- Heat the solid state metal to a high temperature
- Cool it down very slowly according to a specific schedule.
- *If the heating temperature is sufficiently high to ensure random state and the cooling process is slow enough to ensure thermal equilibrium, then the atoms will place themselves in a pattern that corresponds to the global energy minimum of a perfect crystal.*

Simulated Annealing

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state
  inputs: *problem*, a problem
    schedule, a mapping from time to "temperature"

  current ← MAKE-NODE(*problem*.INITIAL-STATE)
  for $t = 1$ **to** ∞ **do**
    $T \leftarrow schedule(t)$
    if $T = 0$ **then return** *current*
    next ← a randomly selected successor of *current*
    $\Delta E \leftarrow next.\text{VALUE} - current.\text{VALUE}$
    if $\Delta E > 0$ **then** *current* ← *next*
    else *current* ← *next* only with probability $e^{\Delta E/T}$

- **Downhill moves** are accepted readily early in the annealing schedule and then less often as time goes on.
- The **schedule** input determines the value of the temperature T as a function of time.

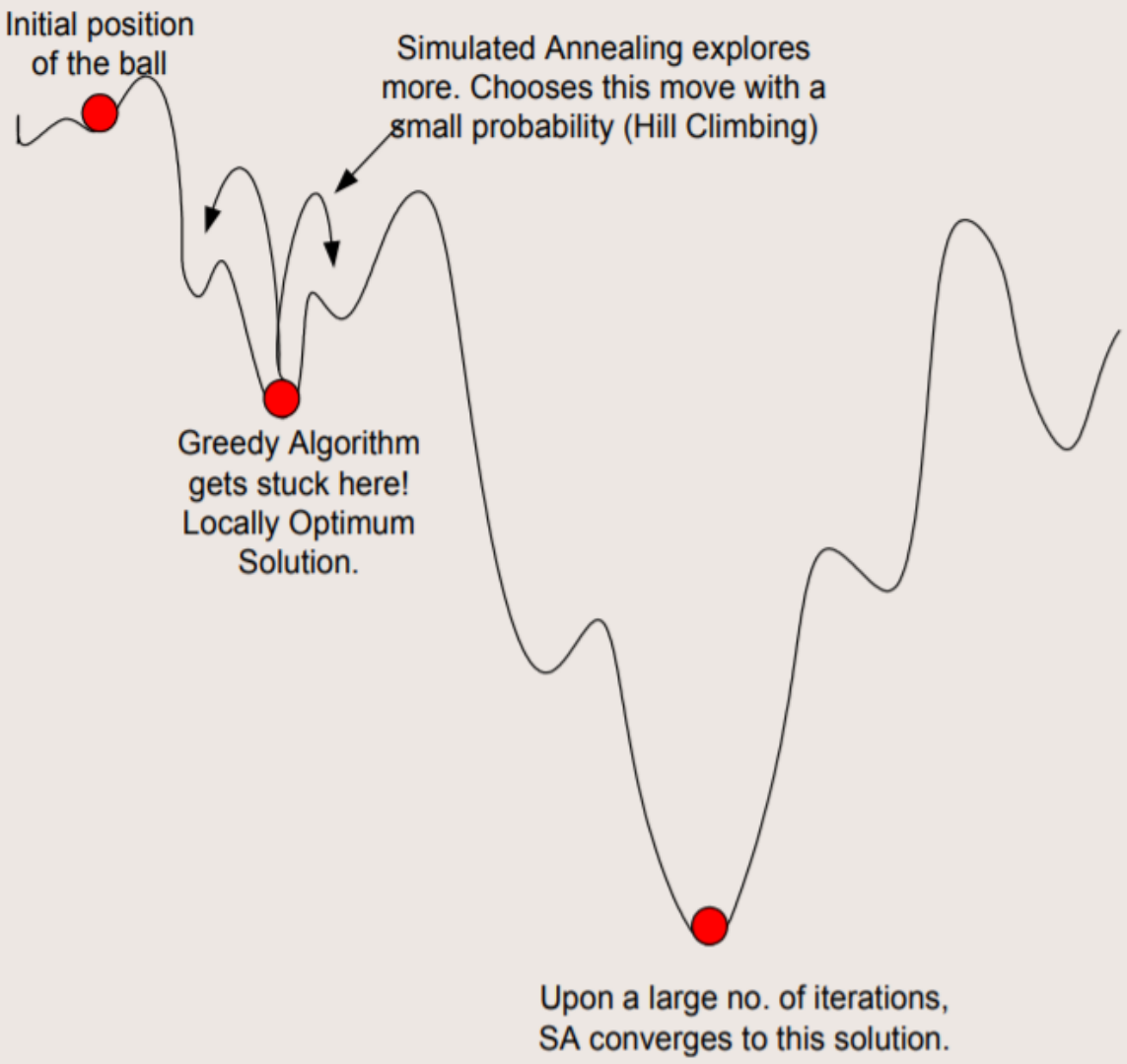
Simulated Annealing: the code

```
1. Create random initial solution γ
2. E_old=cost(γ);
3. for(temp=temp_max; temp>=temp_min;temp=next_temp(temp) ) {
4.     for(i=0;i<i_max; i++ ) {
5.           succesor_func(γ); //this is a randomized function
6.           E_new=cost(γ);
7.           delta=E_new-E_old;
8.           if(delta>0)
9.                 if(random() >= exp(-delta/K*temp);
10.                      undo_func(γ); //rejected bad move
11.                else
12.                      E_old=E_new  //accepted bad move
13.          else
14.                E_old=E_new; //always accept good moves
       }
    }
```

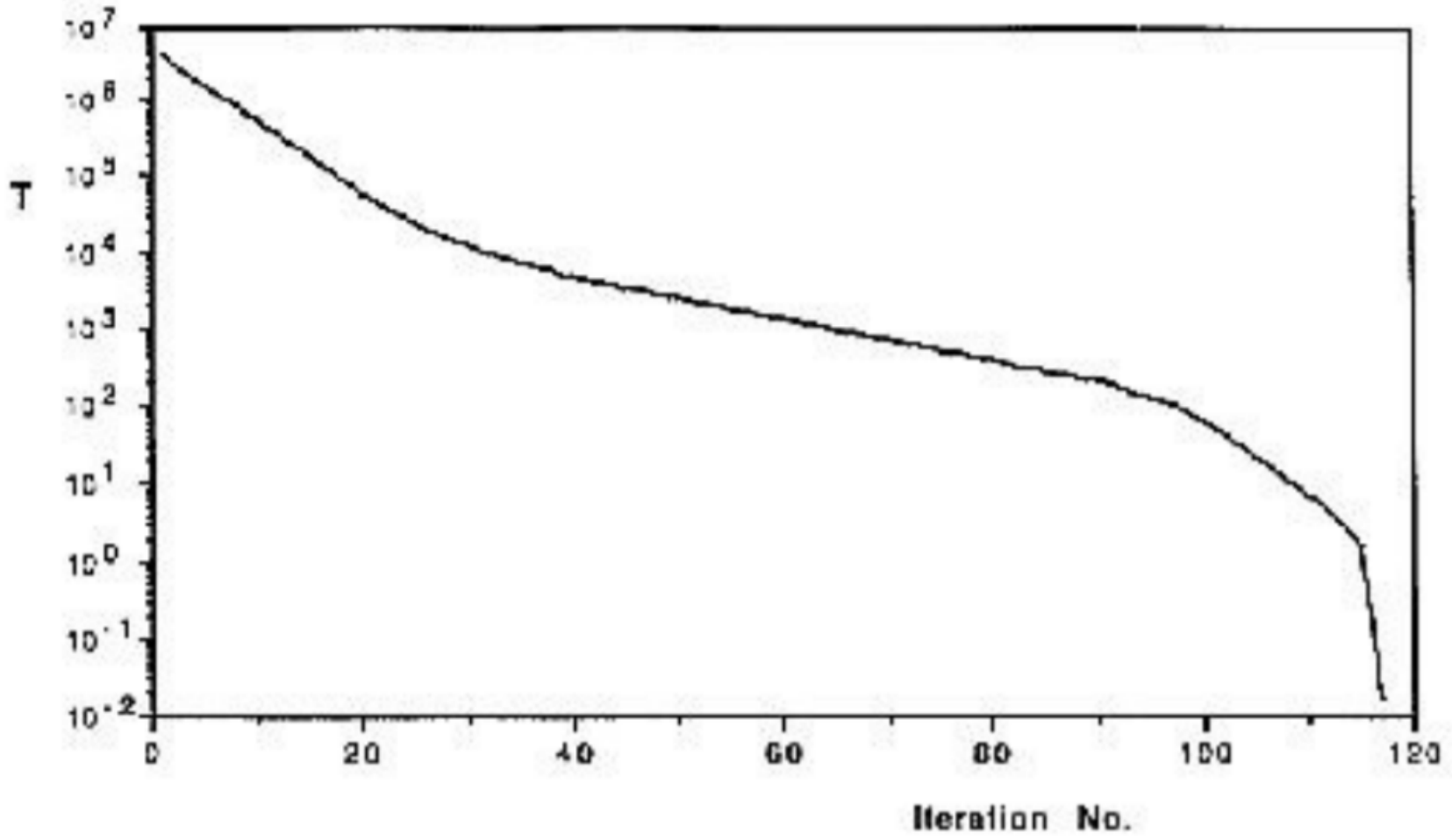
Ball on terrain example – Simulated Annealing vs Greedy Algorithms

The ball is initially placed at a random position on the terrain. From the current position, the ball should be fired such that it can only move one step left or right.What algorithm should we follow for the ball to finally settle at the lowest point on the terrain?

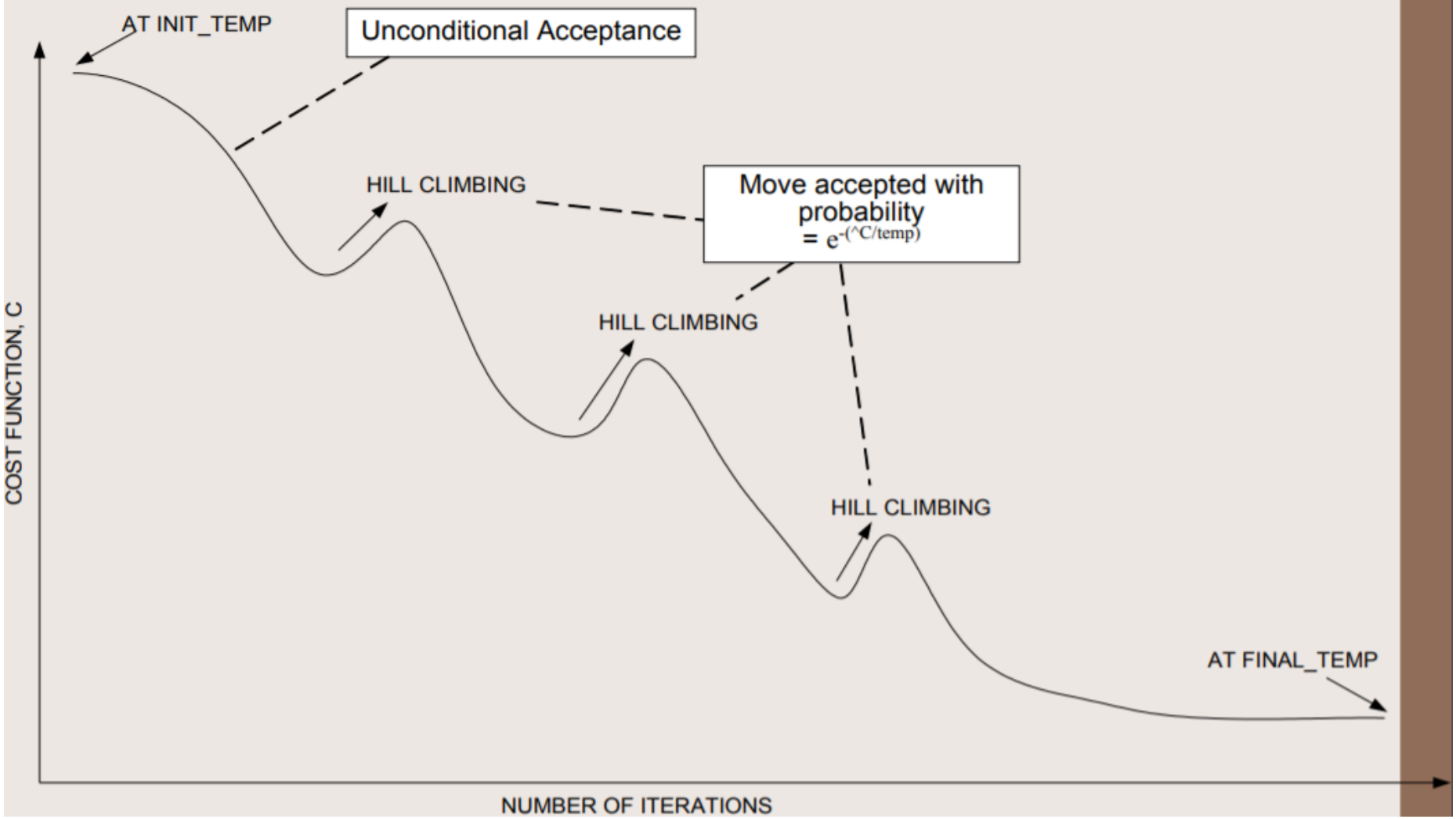
Ball on terrain example – SA vs Greedy Algorithms



Cooling schedule



Convergence of simulated annealing



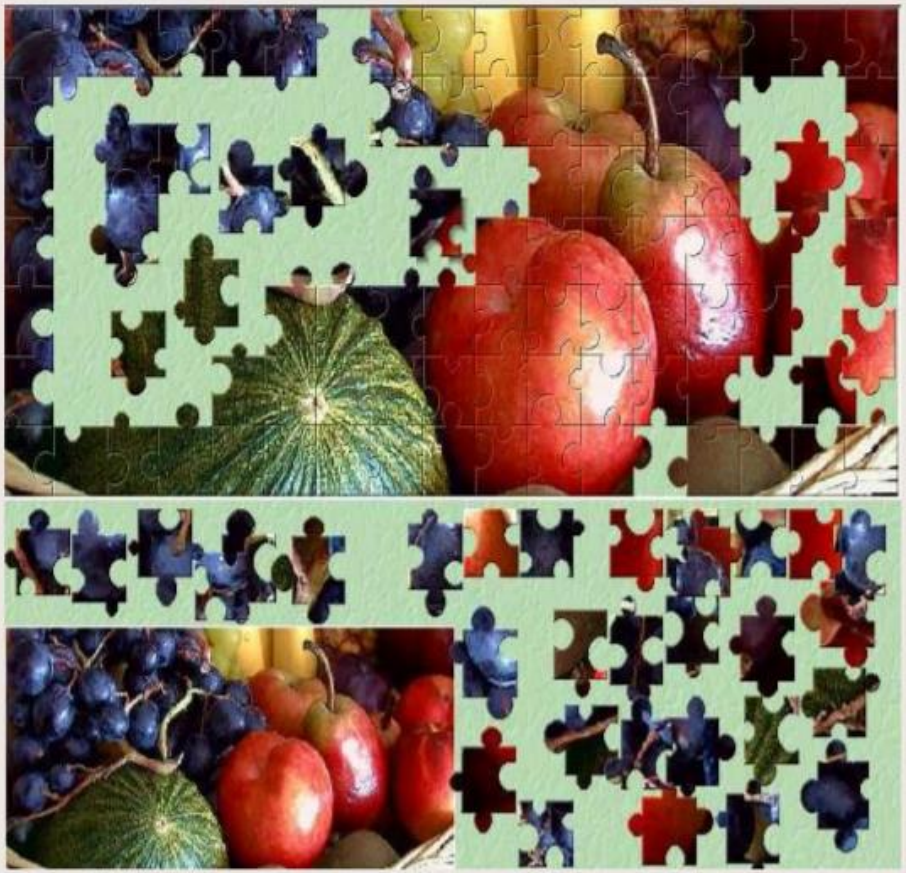
Simulated Annealing

- Instead of picking the best move, Simulated Annealing picks a random move.
  - If the move improves the situation, it is always accepted.
  - Otherwise, the algorithm accepts the move with some probability less than 1.
- The probability decreases exponentially with the "badness" of the move—the amount ΔE by which the evaluation is worsened.
- The probability also decreases as the "temperature" T goes down: "bad" moves are more likely to be allowed at the start when T is high, and they become more unlikely as T decreases.
- **If the schedule lowers T slowly enough, the algorithm will find a best state with probability approaching 1.**

- Simulated Annealing is widely used in VLSI layout and airline scheduling.

Jigsaw puzzles – Intuitive usage of Simulated Annealing

- Given a jigsaw puzzle such that one has to obtain the final shape using all pieces together.
- Starting with a random configuration, the human brain unconditionally chooses certain moves that tend to the solution.
- However, certain moves that may or may not lead to the solution are accepted or rejected with a certain small probability.
- The final shape is obtained as a result of a large number of iterations.



Local Beam Search

- The **local beam search algorithm** keeps track of ***k states rather than just one***.
  - It begins with k randomly generated states.
  - At each step, all the successors of all k states are generated.
  - If any one is a goal, the algorithm halts.
  - Otherwise, it selects the k best successors from the complete list and repeats.

- The **local beam search algorithm** is **not** the same as ***k searches run in parallel***!
  - In a **local beam search**, searches that find good states recruit other searches to join them.
  - In a random-restart search, each search process runs independently of the others.

Stochastic Beam Search

- **Problem:** quite often, all k states end up on same local hill in ***local beam search algorithm***
- **Idea:** choose k successors randomly, biased towards good ones

  ➔ **Stochastic Beam Search**

- Instead of choosing the best k from the pool of candidate successors, **stochastic beam search** chooses k successors at random, with the probability of choosing a given successor being an increasing function of its value.

- **Stochastic beam search** bears some resemblance to the process of natural selection, whereby the "successors" (offspring) of a "state" (organism) populate the next generation according to its "value" (fitness).

Genetic Algorithms

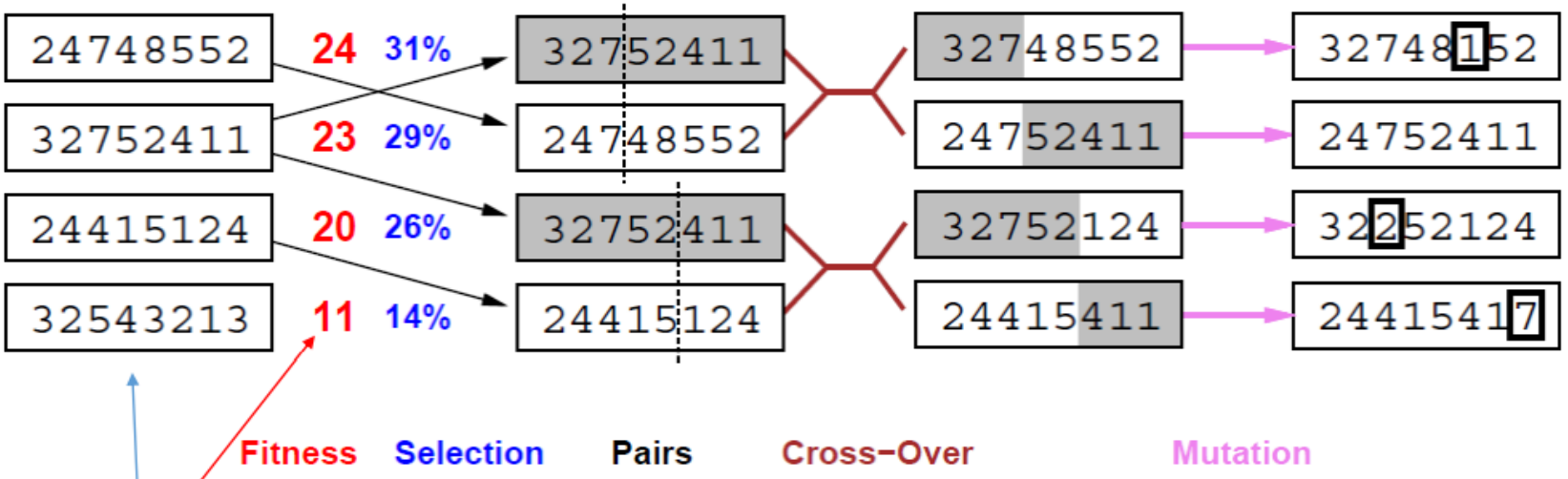
- A **genetic algorithm (GA)** is a variant of ***stochastic beam search*** in which successor states are generated by combining two parent states rather than by modifying a single state.
- Like beam searches, **GAs** begin with a set of ***k randomly generated states***, called the **population**.
- Each state, or **individual**, is represented as a string over a finite alphabet—most commonly, a string of 0s and 1s.
  - An 8-queens state must specify the positions of 8 queens, each in a column of 8 squares, and so it requires $8 \times \log_2 8 = 24$ bits.
  - Alternatively, the state could be represented as 8 digits, each in the range from 1 to 8.
- Each state is rated by an ***objective function***, or (in GA terminology) the **fitness function**.
- Pairs of individuals are randomly selected for ***reproduction***.
- From each pair new offsprings (children) are generated using **crossover** operation.
  - A crossover point is chosen randomly from the positions in the string.
  - The offsprings are created by crossing over the parent strings at the crossover point.
- Each child is subject to random **mutation** with a small independent probability.

Genetic Algorithms

- A state can be represented using a 8 digit string.
- Each digit in the range from 1 to 8 to indicate the position of the queen in that column.

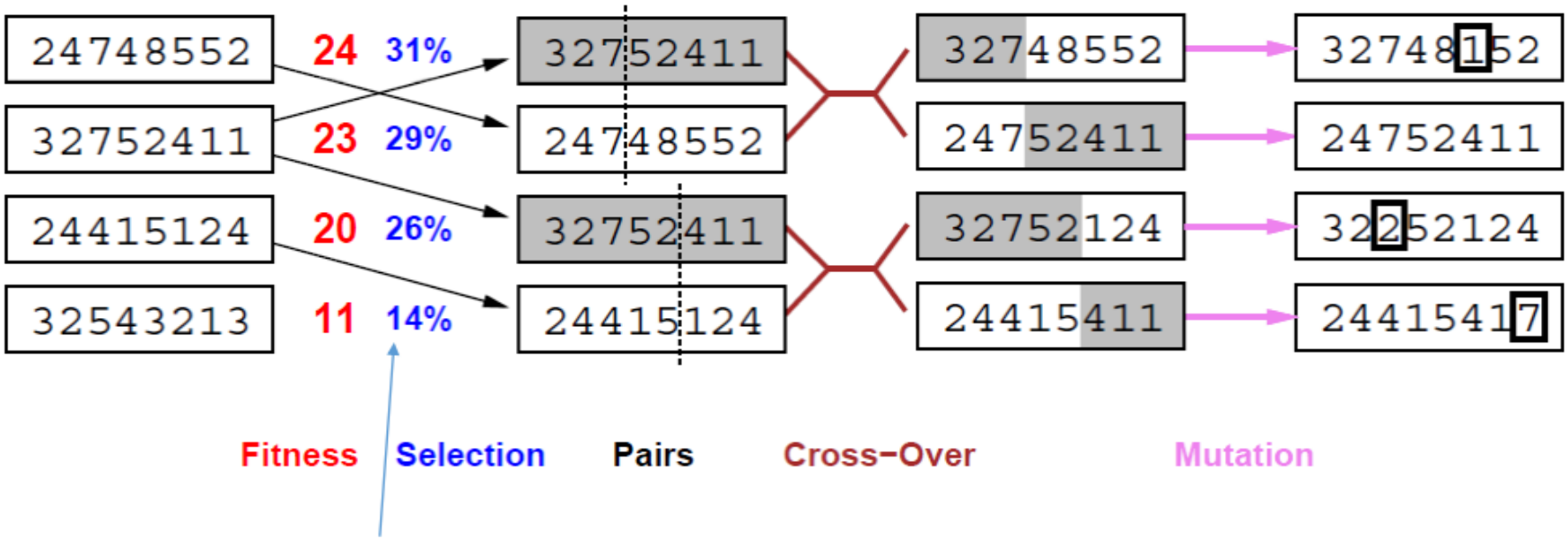
➔ 1 6 2 5 7 4 8 3

Genetic Algorithms



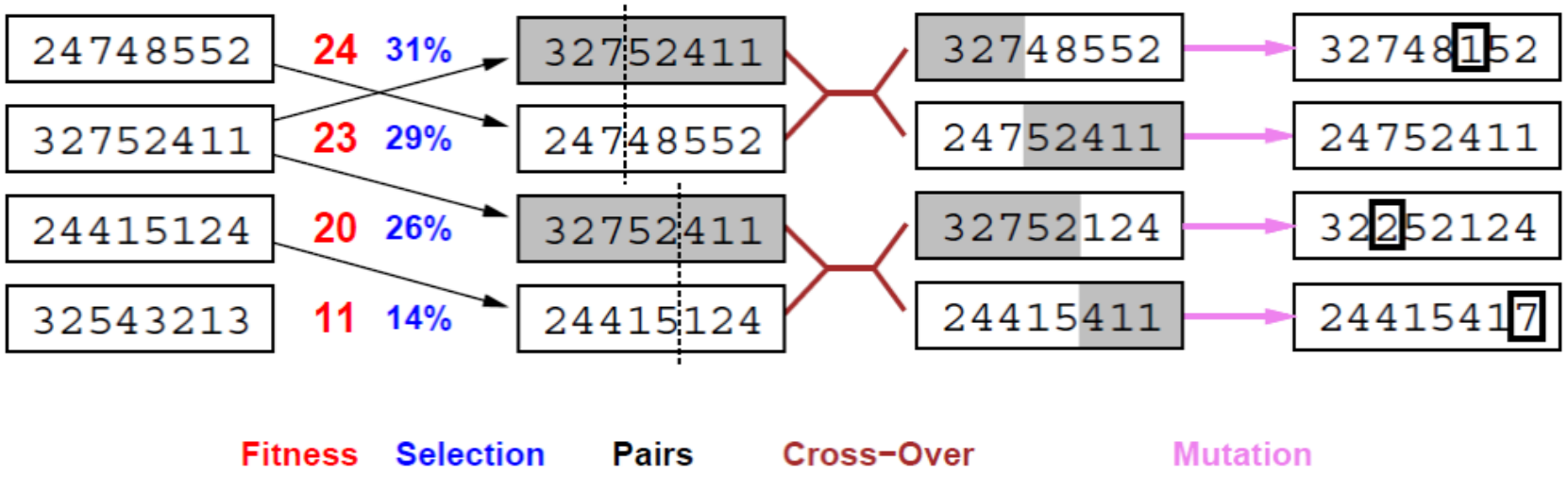
- The **initial population** has 4 states.
- A **fitness function** should return higher values for better states, so, for the 8-queens problem we use the number of non-attacking pairs of queens, which has a value of 28 for a solution.
  - The values of the four states are 24, 23, 20, and 11.

Genetic Algorithms



- The probability of being chosen for reproducing is directly proportional to the fitness score.
- Two pairs are selected at random for reproduction, in accordance with the probabilities.
  - Notice that one individual is selected twice and one not at all.

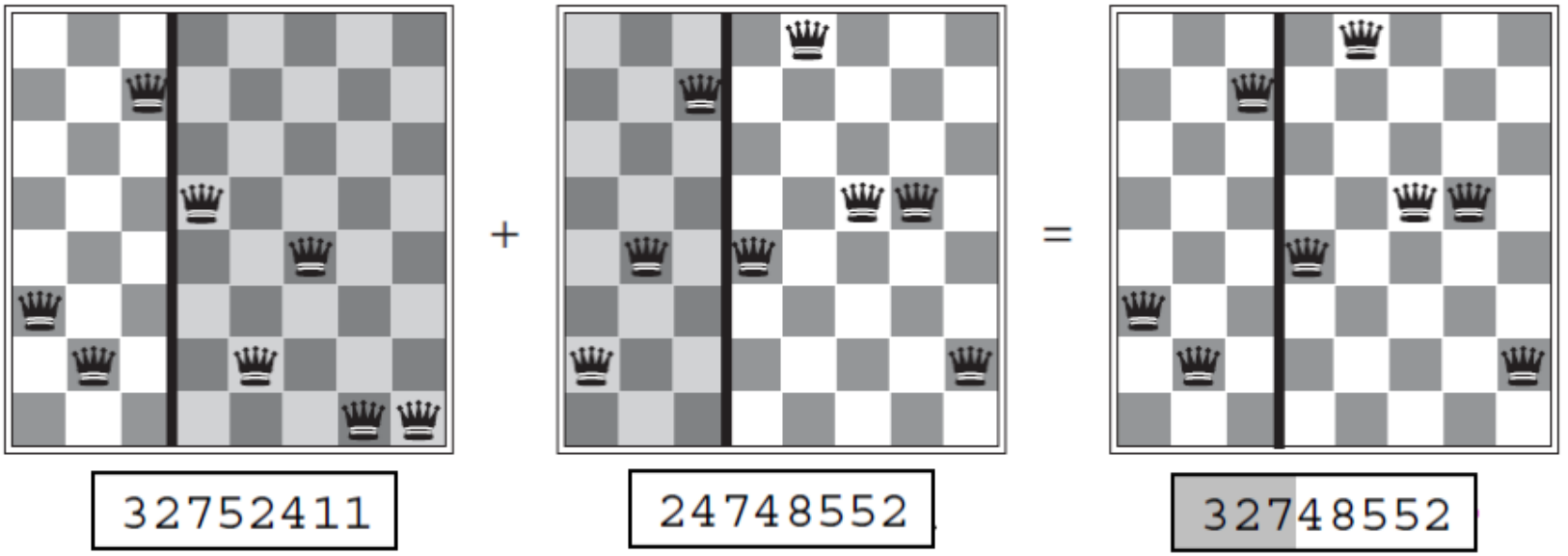
Genetic Algorithms



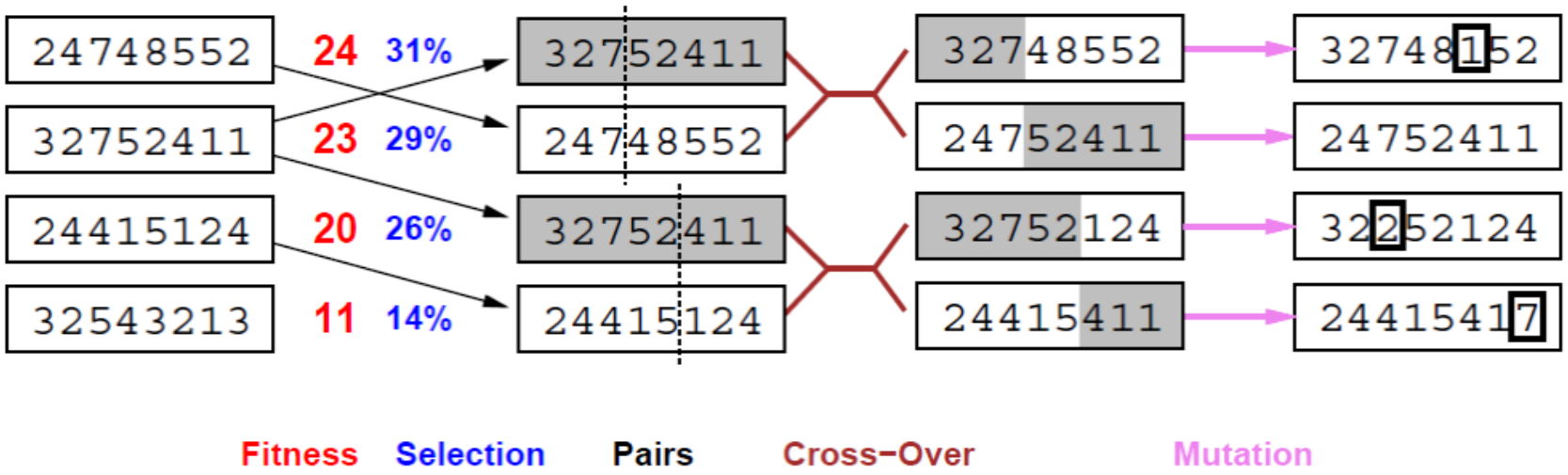
- The **crossover points** are after third digit in first pair and after fifth digit in second pair.
- The first child of the first pair gets the first three digits from the first parent and the remaining digits from the second parent, whereas the second child gets the first three digits from the second parent and the rest from the first parent.

Genetic Algorithms

- Crossover helps iff substrings are meaningful components.
- The shaded columns are lost in the crossover step and the unshaded columns are retained.



Genetic Algorithms



- One digit was **mutated** in the first, third, and fourth offspring.
- In the 8-queens problem, this corresponds to choosing a queen at random and moving it to a random square in its column.

Genetic Algorithms

A genetic algorithm: *each mating of two parents produces only one offspring, not two.*

```
function GENETIC-ALGORITHM(population, FITNESS-FN) returns an individual
  inputs: population, a set of individuals
          FITNESS-FN, a function that measures the fitness of an individual

  repeat
      new_population ← empty set
      for i = 1 to SIZE(population) do
          x ← RANDOM-SELECTION(population, FITNESS-FN)
          y ← RANDOM-SELECTION(population, FITNESS-FN)
          child ← REPRODUCE(x, y)
          if (small random probability) then child ← MUTATE(child)
          add child to new_population
      population ← new_population
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to FITNESS-FN
```

```
function REPRODUCE(x, y) returns an individual
  inputs: x, y, parent individuals

  n ← LENGTH(x); c ← random number from 1 to n
  return APPEND(SUBSTRING(x, 1, c), SUBSTRING(y, c + 1, n))
```

Selection methods in Genetic Algorithm

- **Roulette Wheel Selection**
- **Rank Selection**
- **Steady State Selection**
- **Tournament Selection**
- **Elitism Selection**
- **Boltzmann Selection**

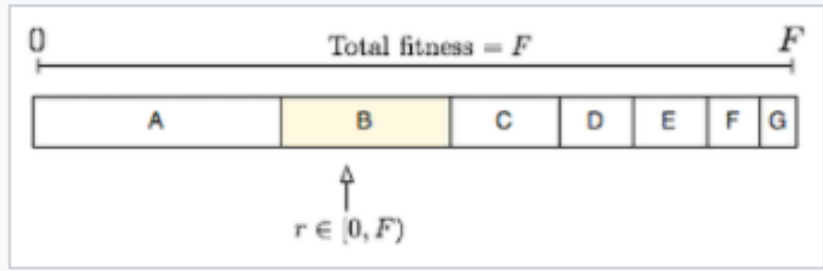
Roulette Wheel Selection

Fitness proportionate selection, also known as **roulette wheel selection**, is a genetic operator used in genetic algorithms for selecting potentially useful solutions for recombination.

In fitness proportionate selection, as in all selection methods, the fitness function assigns a fitness to possible solutions or chromosomes. This fitness level is used to associate a probability of selection with each individual chromosome. If f_i is the fitness of individual i in the population, its probability of being selected is

$$p_i = \frac{f_i}{\sum_{j=1}^{N} f_j},$$

where N is the number of individuals in the population.



Example of the selection of a single individual

This could be imagined similar to a Roulette wheel in a casino. Usually a proportion of the wheel is assigned to each of the possible selections based on their fitness value. This could be achieved by dividing the fitness of a selection by the total fitness of all the selections, thereby normalizing them to 1. Then a random selection is made similar to how the roulette wheel is rotated.

Pseudocode of roulette wheel selection

- For example, if you have a population with fitnesses [1, 2, 3, 4], then the sum is (1 + 2 + 3 + 4 = 10).
- Therefore, you would want the probabilities or chances to be [1/10, 2/10, 3/10, 4/10] or [0.1, 0.2, 0.3, 0.4].
- If you were to visually normalize this between 0.0 and 1.0, it would be grouped like below with [red = 1/10, green = 2/10, blue = 3/10, black = 4/10]:

```
0.1 ]

0.2 \
0.3 /

0.4 \
0.5 |
0.6 /

0.7 \
0.8 |
0.9 |
1.0 /
```

Using the above example numbers, this is how to determine the probabilities:

```
sum_of_fitness = 10
previous_probability = 0.0

[1] = previous_probability + (fitness / sum_of_fitness) = 0.0 + (1 / 10) = 0.1
previous_probability = 0.1

[2] = previous_probability + (fitness / sum_of_fitness) = 0.1 + (2 / 10) = 0.3
previous_probability = 0.3

[3] = previous_probability + (fitness / sum_of_fitness) = 0.3 + (3 / 10) = 0.6
previous_probability = 0.6

[4] = previous_probability + (fitness / sum_of_fitness) = 0.6 + (4 / 10) = 1.0
```

The last index should always be 1.0 or close to it. Then this is how to randomly select an individual:

```
random_number # Between 0.0 and 1.0

if random_number < 0.1
    select 1
else if random_number < 0.3 # 0.3 - 0.1 = 0.2 probability
    select 2
else if random_number < 0.6 # 0.6 - 0.3 = 0.3 probability
    select 3
else if random_number < 1.0 # 1.0 - 0.6 = 0.4 probability
    select 4
end if
```

Elitism Selection

- Often to get better parameters, strategies with partial reproduction are used.
- One of them is elitism, in which a small portion of the best individuals from the last generation is carried over (without any changes) to the next one.

Reinsertion (Replacement)

- Once the offspring have been produced by selection, recombination and mutation of individuals from the old population, the fitness of the offspring may be determined.
- If less offspring are produced than the size of the original population then to maintain the size of the original population, the offspring have to be reinserted into the old population.
- Similarly, if not all offspring are to be used at each generation or if more offspring are generated than the size of the old population then a reinsertion scheme must be used to determine which individuals are to exist in the new population.

Reinsertion

- Different schemes of global reinsertion exist:

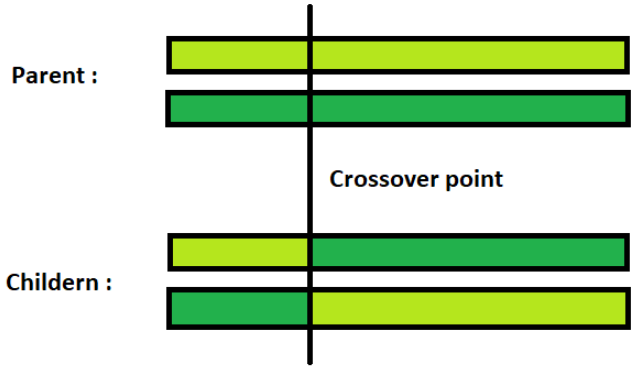
- produce as many offspring as parents and replace all parents by the offspring (pure reinsertion).
- produce less offspring than parents and replace parents uniformly at random (uniform reinsertion).
- produce less offspring than parents and replace the worst parents (elitist reinsertion).
- produce more offspring than needed for reinsertion and reinsert only the best offspring (fitness-based reinsertion).

Reinsertion

- Pure Reinsertion is the simplest reinsertion scheme. Every individual lives one generation only. This scheme is used in the simple genetic algorithm. However, it is very likely, that very good individuals are replaced without producing better offspring and thus, good information is lost.
- The elitist combined with fitness-based reinsertion prevents this losing of information and is the recommended method. At each generation, a given number of the least fit parents is replaced by the same number of the most fit offspring.

Different types of crossover

- In genetic algorithms and evolutionary computation, crossover, also called recombination, is a genetic operator used to combine the genetic information of two parents to generate new offspring.
- **Single Point Crossover :** A crossover point on the parent organism string is selected. All data beyond that point in the organism string is swapped between the two parent organisms. Strings are characterized by Positional Bias.



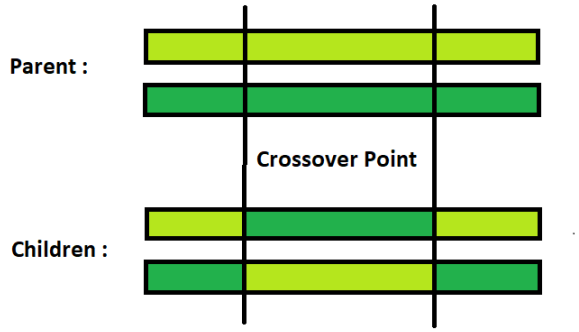
Different types of crossover

Chromosome1	11011|00100110110
Chromosome2	11011|11000011110
Offspring1	11011|11000011110
Offspring2	11011|00100110110

Single Point Crossover

Different types of crossover

- **Two-Point Crossover :** This is a specific case of a N-point Crossover technique. Two random points are chosen on the individual chromosomes (strings) and the genetic material is exchanged at these points.

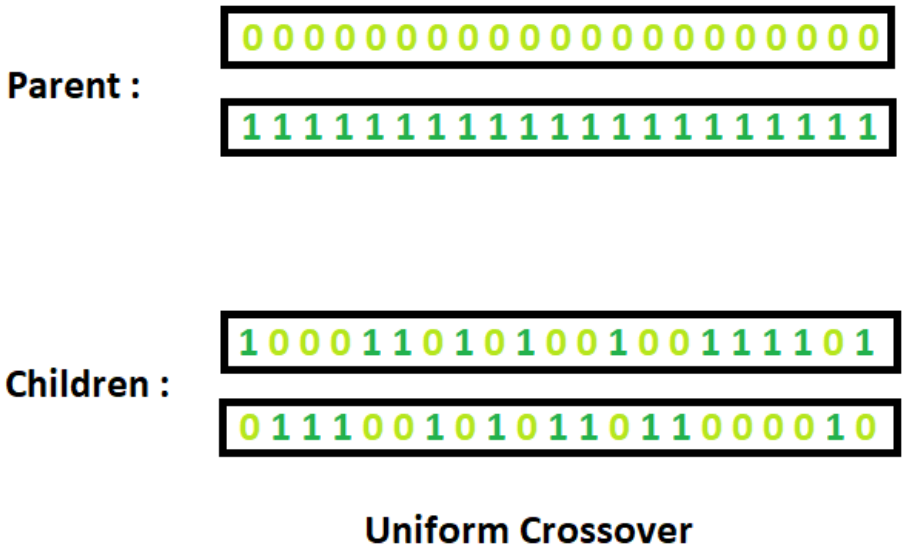


Chromosome1	11011|00100|110110
Chromosome2	10101|11000|011110
Offspring1	11011|11000|110110
Offspring2	10101|00100|011110

Two Point Crossover

Different types of crossover

- **Uniform Crossover :** Each gene (bit) is selected randomly from one of the corresponding genes of the parent chromosomes.
- Use tossing of a coin as an example technique.



Uniform Crossover

Different types of crossover

- **Crossover for ordered lists:** In some genetic algorithms, not all possible chromosomes represent valid solutions. In some cases, it is possible to use specialized crossover and mutation operators that are designed to avoid violating the constraints of the problem.

- For example, a genetic algorithm solving the travelling salesman problem may use an ordered list of cities to represent a solution path.
- Such a chromosome only represents a valid solution if the list contains all the cities that the salesman must visit.
- Using the above crossovers will often result in chromosomes that violate that constraint.
- Genetic algorithms optimizing the ordering of a given list thus require different crossover operators that will avoid generating invalid solutions. Many such crossovers have been published:

Different types of crossover

- partially mapped crossover (PMX)
- cycle crossover (CX)
- order crossover operator (OX1)
- order-based crossover operator (OX2)
- position-based crossover operator (POS)
- voting recombination crossover operator (VR)
- alternating-position crossover operator (AP)
- sequential constructive crossover operator (SCX)
- simulated binary crossover operator (SBX)

Genetic Algorithm Parameters

- Determining the interactions that occur among different GA parameters has a direct impact on the quality of the solution, and keeping parameters values "balanced" improves the solution of the GA.
- There are four basic and important parameters used by GA, those include:

1) Crossover rate (probability): the number of times a crossover occurs for chromosomes in one generation, i.e., the chance that two chromosomes exchange some of their parts), 100% crossover rate means that all offspring are made by crossover. If it is 0%, then the complete new generation

of individuals is to be exactly copied from the older population, except those resulted from the mutation process. Crossover rate is in the range of [0, 1].

Genetic Algorithm Parameters

2) Mutation rate (probability): this rate determines how many chromosomes should be mutated in one generation; mutation rate is in the range of [0, 1]. The purpose of mutation is to prevent the GA from converging to local optima, but if it occurs very often, GA is changed to random search.

3) Population size: the size of the population indicates the total number of the population's inhabitants. Selection of population size is a sensitive issue; if the size of the population (search space) is small, this means little search space is available, and therefore it is possible to reach a local optimum. although, if the population size is very large, the area of search is increased and the computational load becomes high, therefore, the size of the population must be reasonable.

Genetic Algorithm Parameters

4) Number of generations: It refers to the number of cycles before the termination. In some cases, hundreds of loops are sufficient, but in other cases we might need more, this depends on the problem type and complexity. Depending on the design of the GA, sometimes this parameter is not used, particularly if the termination of the GA depends on specific criteria.

These GA parameters are determined whether the GA finds an optimal or near-optimal solution, and whether it finds an efficient solution. Any change in the value of these parameters (increasing or decreasing) affects the result of GA negatively or positively.

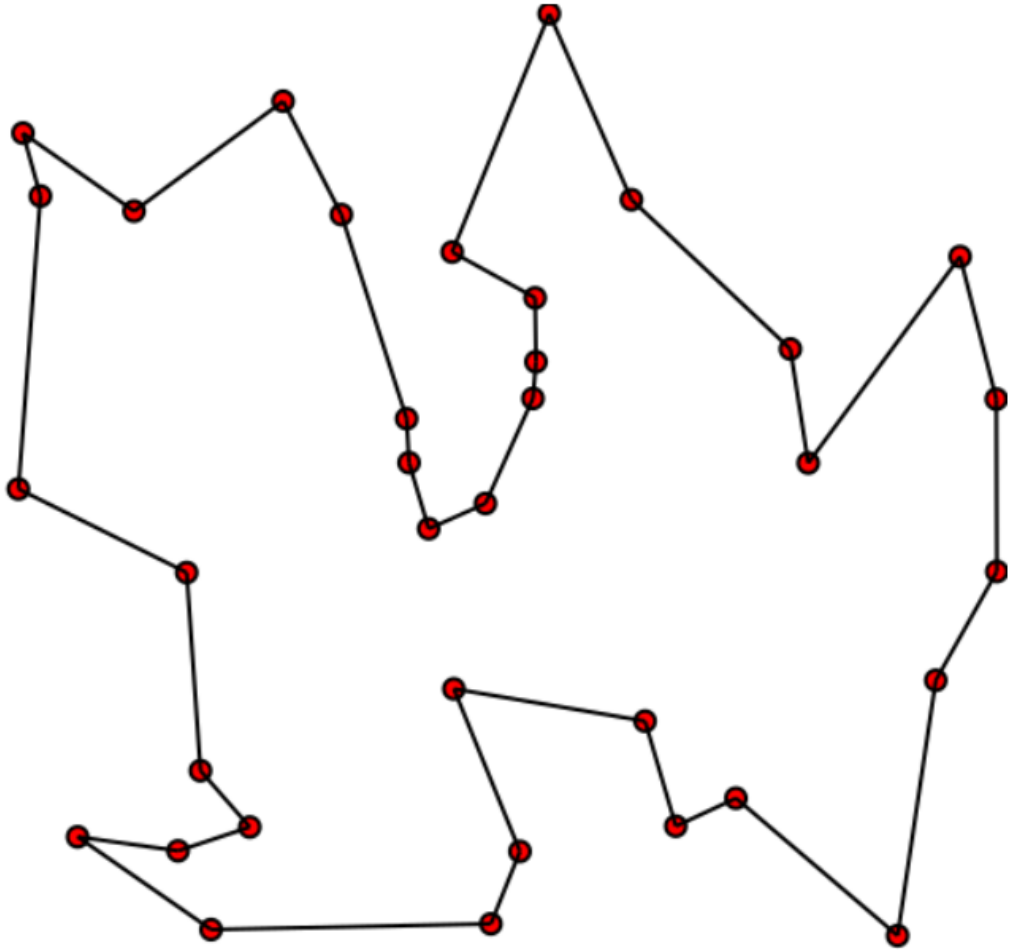
Knapsack problem solving with Genetic Algorithm

- https://www.youtube.com/watch?v=JgqBM7JG9ew

The problem

In this tutorial, we'll be using a GA to find a solution to the traveling salesman problem (TSP). The TSP is described as follows:

> *"Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city?"*



Given this, there are two important rules to keep in mind:

1. Each city needs to be visited exactly one time
2. We must return to the starting city, so our total distance needs to be calculated accordingly

The approach

Let's start with a few definitions, rephrased in the context of the TSP:

- **Gene:** a city (represented as (x, y) coordinates)
- **Individual (aka "chromosome"):** a single route satisfying the conditions above
- **Population:** a collection of possible routes (i.e., collection of individuals)
- **Parents:** two routes that are combined to create a new route
- **Mating pool:** a collection of parents that are used to create our next population (thus creating the next generation of routes)
- **Fitness:** a function that tells us how good each route is (in our case, how short the distance is)
- **Mutation:** a way to introduce variation in our population by randomly swapping two cities in a route
- **Elitism:** a way to carry the best individuals into the next generation

- Our GA will proceed in the following steps:

1. Create the population
2. Determine fitness
3. Select the mating pool
4. Breed
5. Mutate
6. Repeat

Building our genetic algorithm

While each part of our GA is built from scratch, we'll use a few standard packages to make things easier:

```
import numpy as np, random, operator, pandas as pd, matplotlib.pyplot
as plt
```

Create two classes: City and Fitness

We first create a `City` class that will allow us to create and handle our cities. These are simply our (x, y) coordinates. Within the City class, we add a `distance` calculation (making use of the Pythagorean theorem) in line 6 and a cleaner way to output the cities as coordinates with `__repr__` in line 12.

```
1   class City:
2       def __init__(self, x, y):
3           self.x = x
4           self.y = y
5
6       def distance(self, city):
7           xDis = abs(self.x - city.x)
8           yDis = abs(self.y - city.y)
9           distance = np.sqrt((xDis ** 2) + (yDis ** 2))
10          return distance
11
12      def __repr__(self):
13          return "(" + str(self.x) + "," + str(self.y) + ")"
```

genetic_algo_1.py hosted with ❤ by GitHub view raw

We'll also create a `Fitness` class. In our case, we'll treat the fitness as the inverse of the route distance. We want to minimize route distance, so a larger fitness score is better. Based on Rule #2, we need to start and end at the same place, so this extra calculation is accounted for in line 13 of the distance calculation.

```
1  class Fitness:
2      def __init__(self, route):
3          self.route = route
4          self.distance = 0
5          self.fitness= 0.0
6
7      def routeDistance(self):
8          if self.distance ==0:
9              pathDistance = 0
10             for i in range(0, len(self.route)):
11                 fromCity = self.route[i]
12                 toCity = None
13                 if i + 1 < len(self.route):
14                     toCity = self.route[i + 1]
15                 else:
16                     toCity = self.route[0]
17                 pathDistance += fromCity.distance(toCity)
18             self.distance = pathDistance
19         return self.distance
20
21     def routeFitness(self):
22         if self.fitness == 0:
23             self.fitness = 1 / float(self.routeDistance())
24         return self.fitness
```

Create the population

We now can make our initial population (aka first generation). To do so, we need a way to create a function that produces routes that satisfy our conditions (*Note: we'll create our list of cities when we actually run the GA at the end of the tutorial*). To create an individual, we randomly select the order in which we visit each city:

```
def createRoute(cityList):
    route = random.sample(cityList, len(cityList))
    return route
```

genetic_algo_3.py hosted with ❤ by GitHub view raw

This produces one individual, but we want a full population, so let's do that in our next function. This is as simple as looping through the `createRoute` function until we have as many routes as we want for our population.

```
def initialPopulation(popSize, cityList):
    population = []

    for i in range(0, popSize):
        population.append(createRoute(cityList))
    return population
```

Determine fitness

Next, the evolutionary fun begins. To simulate our "survival of the fittest", we can make use of `Fitness` to rank each individual in the population. Our output will be an ordered list with the route IDs and each associated fitness score.

```python
def rankRoutes(population):
    fitnessResults = {}
    for i in range(0,len(population)):
        fitnessResults[i] = Fitness(population[i]).routeFitness()
    return sorted(fitnessResults.items(), key = operator.itemgetter(1), reverse = True)
```

genetic_algo_5.py hosted with ❤ by GitHub view raw

Select the mating pool

There are a few options for how to select the parents that will be used to create the next generation. The most common approaches are either **fitness proportionate selection** (aka "roulette wheel selection") or **tournament selection**:

- **Fitness proportionate selection** (the version implemented below): The fitness of each individual relative to the population is used to assign a probability of selection. Think of this as the fitness-weighted probability of being selected.
- **Tournament selection**: A set number of individuals are randomly selected from the population and the one with the highest fitness in the group is chosen as the first parent. This is repeated to chose the second parent.

Another design feature to consider is the use of **elitism**. With elitism, the best performing individuals from the population will automatically carry over to the next generation, ensuring that the most successful individuals persist.

For the purpose of clarity, we'll create the mating pool in two steps. First, we'll use the output from `rankRoutes` to determine which routes to select in our `selection` function. In lines 3–5, we set up the roulette wheel by calculating a relative fitness weight for each individual. In line 9, we compare a randomly drawn number to these weights to select our mating pool. We'll also want to hold on to our best routes, so we introduce elitism in line 7. Ultimately, the `selection` function returns a list of route IDs, which we can use to create the mating pool in the `matingPool` function.

```
1  def selection(popRanked, eliteSize):
2      selectionResults = []
3      df = pd.DataFrame(np.array(popRanked), columns=["Index","Fitness"])
4      df['cum_sum'] = df.Fitness.cumsum()
5      df['cum_perc'] = 100*df.cum_sum/df.Fitness.sum()
6
7      for i in range(0, eliteSize):
8          selectionResults.append(popRanked[i][0])
9      for i in range(0, len(popRanked) - eliteSize):
10         pick = 100*random.random()
11         for i in range(0, len(popRanked)):
12             if pick <= df.iat[i,3]:
13                 selectionResults.append(popRanked[i][0])
14                 break
15     return selectionResults
```

Now that we have the IDs of the routes that will make up our mating pool from the `selection` function, we can create the mating pool. We're simply extracting the selected individuals from our population.

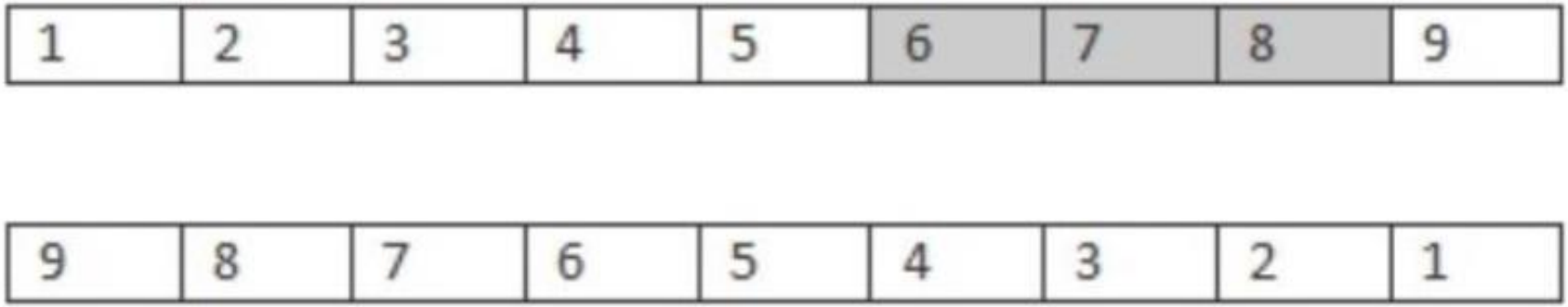
```
def matingPool(population, selectionResults):
    matingpool = []
    for i in range(0, len(selectionResults)):
        index = selectionResults[i]
        matingpool.append(population[index])
    return matingpool
```

Breed

With our mating pool created, we can create the next generation in a process called **crossover** (aka "breeding"). If our individuals were strings of 0s and 1s and our two rules didn't apply (e.g., imagine we were deciding whether or not to include a stock in a portfolio), we could simply pick a crossover point and splice the two strings together to produce an offspring.

However, the TSP is unique in that we need to include all locations exactly one time. To abide by this rule, we can use a special breeding function called **ordered crossover**. In ordered crossover, we randomly select a subset of the first parent string (see line 12 in `breed` function below) and then fill the remainder of the route with the genes from the second parent in the order in which they appear, without duplicating any genes in the selected subset from the first parent (see line 15 in `breed` function below).

Parents



Offspring

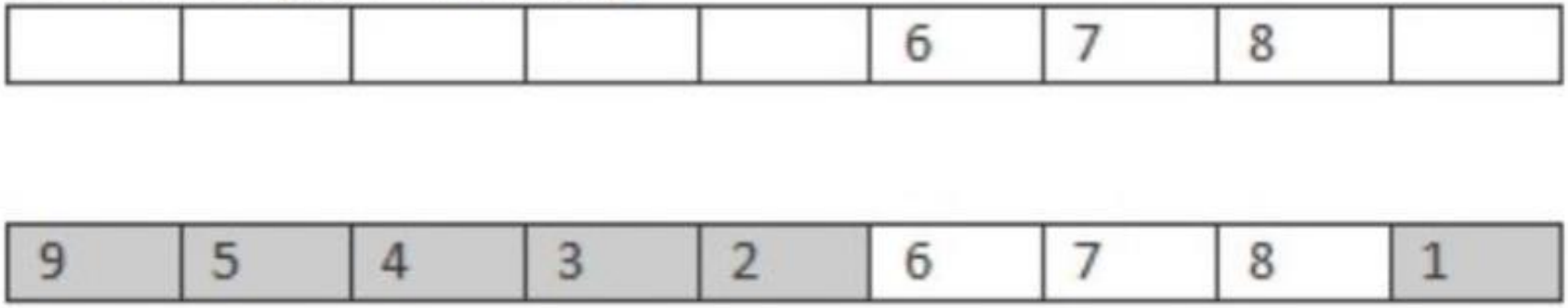


Illustration of ordered crossover

```python
def breed(parent1, parent2):
    child = []
    childP1 = []
    childP2 = []

    geneA = int(random.random() * len(parent1))
    geneB = int(random.random() * len(parent1))

    startGene = min(geneA, geneB)
    endGene = max(geneA, geneB)

    for i in range(startGene, endGene):
        childP1.append(parent1[i])

    childP2 = [item for item in parent2 if item not in childP1]

    child = childP1 + childP2
    return child
```

Next, we'll generalize this to create our offspring population. In line 5, we use elitism to retain the best routes from the current population. Then, in line 8, we use the `breed` function to fill out the rest of the next generation.

```
1  def breedPopulation(matingpool, eliteSize):
2      children = []
3      length = len(matingpool) - eliteSize
4      pool = random.sample(matingpool, len(matingpool))
5
6      for i in range(0,eliteSize):
7          children.append(matingpool[i])
8
9      for i in range(0, length):
10         child = breed(pool[i], pool[len(matingpool)-i-1])
11         children.append(child)
12     return children
```

genetic_algo_9.py hosted with ❤ by GitHub view raw

Mutate

Mutation serves an important function in GA, as it helps to avoid local convergence by introducing novel routes that will allow us to explore other parts of the solution space. Similar to crossover, the TSP has a special consideration when it comes to mutation. Again, if we had a chromosome of 0s and 1s, mutation would simply mean assigning a low probability of a gene changing from 0 to 1, or vice versa (to continue the example from before, a stock that was included in the offspring portfolio is now excluded).

However, since we need to abide by our rules, we can't drop cities. Instead, we'll use **swap mutation**. This means that, with specified low probability, two cities will swap places in our route. We'll do this for one individual in our `mutate` function:

```
def mutate(individual, mutationRate):
    for swapped in range(len(individual)):
        if(random.random() < mutationRate):
            swapWith = int(random.random() * len(individual))

            city1 = individual[swapped]
            city2 = individual[swapWith]

            individual[swapped] = city2
            individual[swapWith] = city1
    return individual
```

Repeat

We're almost there. Let's pull these pieces together to create a function that produces a new generation. First, we rank the routes in the current generation using `rankRoutes`. We then determine our potential parents by running the `selection` function, which allows us to create the mating pool using the `matingPool` function. Finally, we then create our new generation using the `breedPopulation` function and then applying mutation using the `mutatePopulation` function.

```
def nextGeneration(currentGen, eliteSize, mutationRate):
    popRanked = rankRoutes(currentGen)
    selectionResults = selection(popRanked, eliteSize)
    matingpool = matingPool(currentGen, selectionResults)
    children = breedPopulation(matingpool, eliteSize)
    nextGeneration = mutatePopulation(children, mutationRate)
    return nextGeneration
```

Evolution in motion

We finally have all the pieces in place to create our GA! All we need to do is create the initial population, and then we can loop through as many generations as we desire. Of course we also want to see the best route and how much we've improved, so we capture the initial distance in line 3 (remember, distance is the inverse of the fitness), the final distance in line 8, and the best route in line 9.

```
1  def geneticAlgorithm(population, popSize, eliteSize, mutationRate, generations):
2      pop = initialPopulation(popSize, population)
3      print("Initial distance: " + str(1 / rankRoutes(pop)[0][1]))
4
5      for i in range(0, generations):
6          pop = nextGeneration(pop, eliteSize, mutationRate)
7
8      print("Final distance: " + str(1 / rankRoutes(pop)[0][1]))
9      bestRouteIndex = rankRoutes(pop)[0][0]
10     bestRoute = pop[bestRouteIndex]
11     return bestRoute
```

Running the genetic algorithm

With everything in place, solving the TSP is as easy as two steps:

First, we need a list of cities to travel between. For this demonstration, we'll create a list of 25 random cities (a seemingly small number of cities, but brute force would have to test over 300 sextillion routes!):

```
cityList = []

for i in range(0,25):
    cityList.append(City(x=int(random.random() * 200), y=int(random.random() * 200)))
```

Then, running the genetic algorithm is one simple line of code. This is where art meets science; you should see which assumptions work best for you. In this example, we have 100 individuals in each generation, keep 20 elite individuals, use a 1% mutation rate for a given gene, and run through 500 generations:

```
1   geneticAlgorithm(population=cityList, popSize=100, eliteSize=20, mutationRate=0.01, generations=5
```

Bonus feature: Plot the improvement

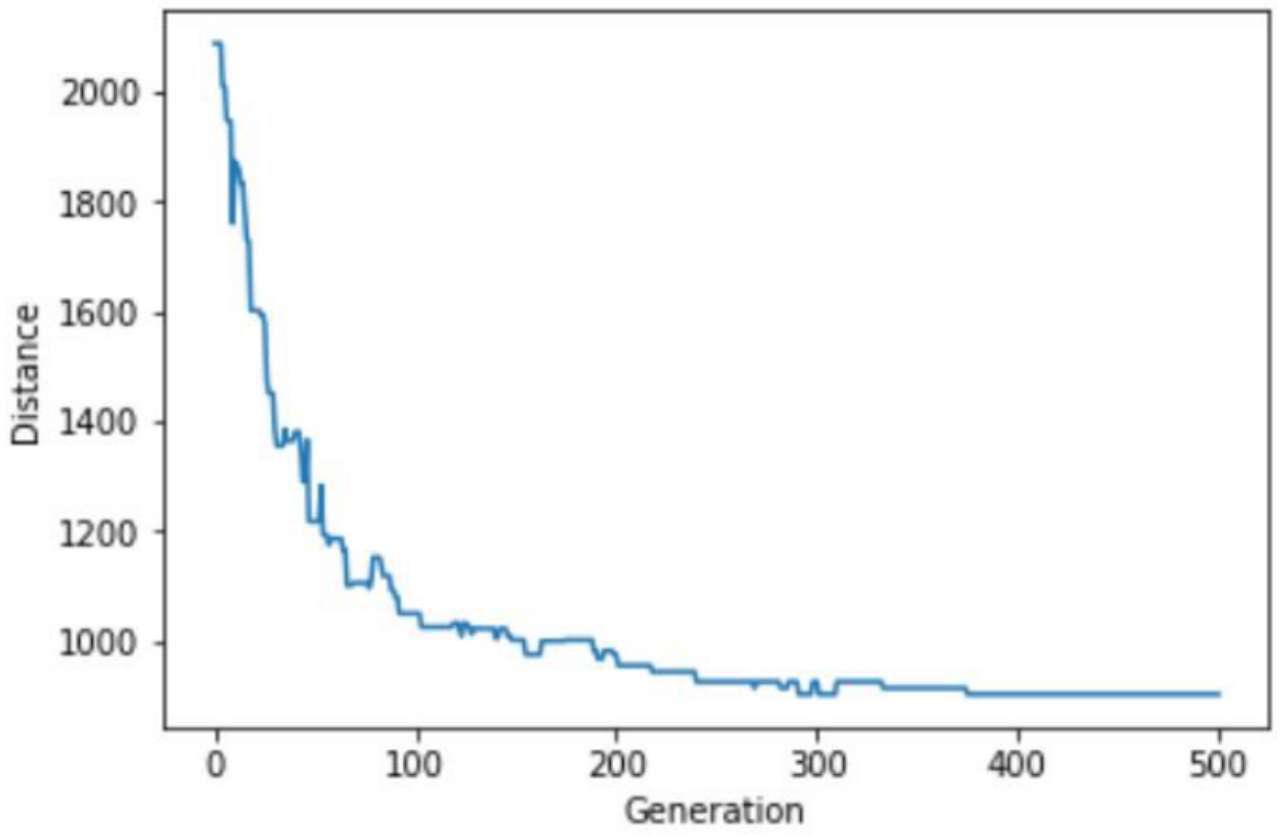
It's great to know our starting and ending distance and the proposed route, but we would be remiss not to see how our distance improved over time. With a simple tweak to our `geneticAlgorithm` function, we can store the shortest distance from each generation in a `progress` list and then plot the results.

```
def geneticAlgorithmPlot(population, popSize, eliteSize, mutationRate, generations):
    pop = initialPopulation(popSize, population)
    progress = []
    progress.append(1 / rankRoutes(pop)[0][1])

    for i in range(0, generations):
        pop = nextGeneration(pop, eliteSize, mutationRate)
        progress.append(1 / rankRoutes(pop)[0][1])

    plt.plot(progress)
    plt.ylabel('Distance')
    plt.xlabel('Generation')
    plt.show()
```

Run the GA in the same way as before, but now using the newly created `geneticAlgorithmPlot` function:

```
1  geneticAlgorithmPlot(population=cityList, popSize=100, eliteSize=20, mutationRate=0.01, generatio
```

genetic_algo_17.py hosted with ❤ by GitHub view raw



Sample output from the geneticAlgorithmPlot function

Particle Swarm Optimization (PSO)

- PSO is originally attributed to Kennedy, Eberhart and Shi and was first intended for simulating social behaviour, as a stylized representation of the movement of organisms in a bird flock or fish school.
- Suppose a group of birds is searching food in an area
- Only one piece of food is available
- Birds do not have any knowledge about the location of the food

PSO

- But they know how far the food is from their present location
- So what is the best strategy to locate the food?
- The best strategy is to follow the bird nearest to the food

PSO

Current position

Next position

A flying bird has a position and a velocity at any time *t*

In search of food, the bird changes his position by adjusting the velocity

The velocity changes based on his past experience and also the feedbacks received from his neighbor

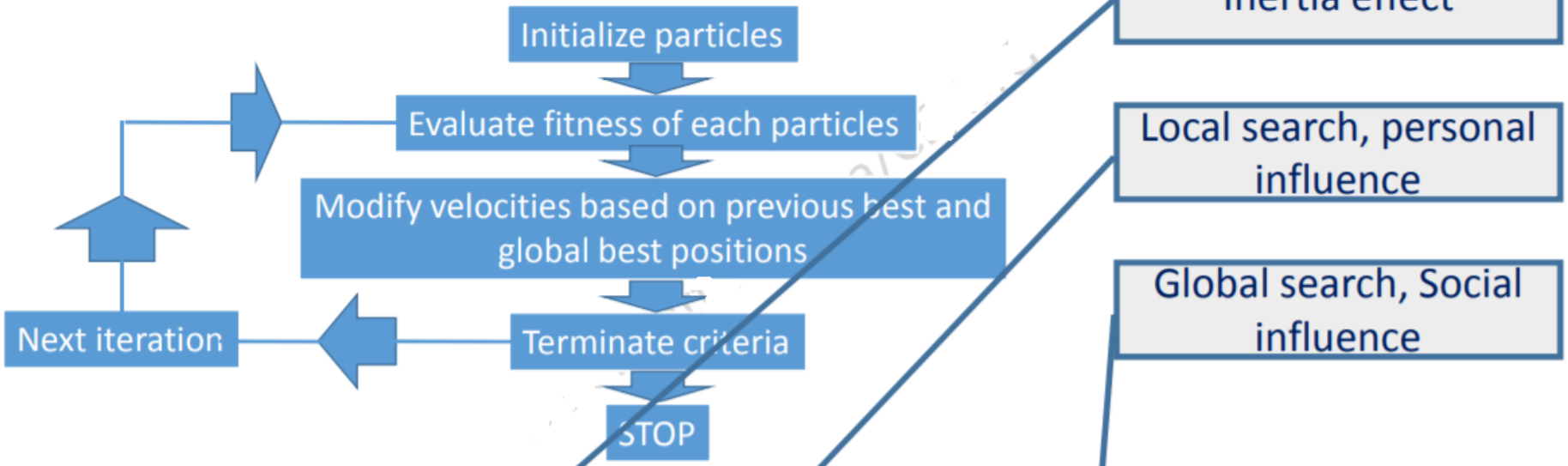
This searching process can be artificially simulated for solving non-linear optimization problem

So this is a population based stochastic optimization technique inspired by social behaviour of bird flocking or fish schooling

PSO

- Each solution is considered as bird, called particle
- All the particles have a fitness value.
- The fitness values can be calculated using objective function
- All the particles preserve their individual best performance
- They also know the best performance of their group
- They adjust their velocity considering their best performance and also considering the best performance of the best particle

Particle Swarm Algorithm



Velocity is updated

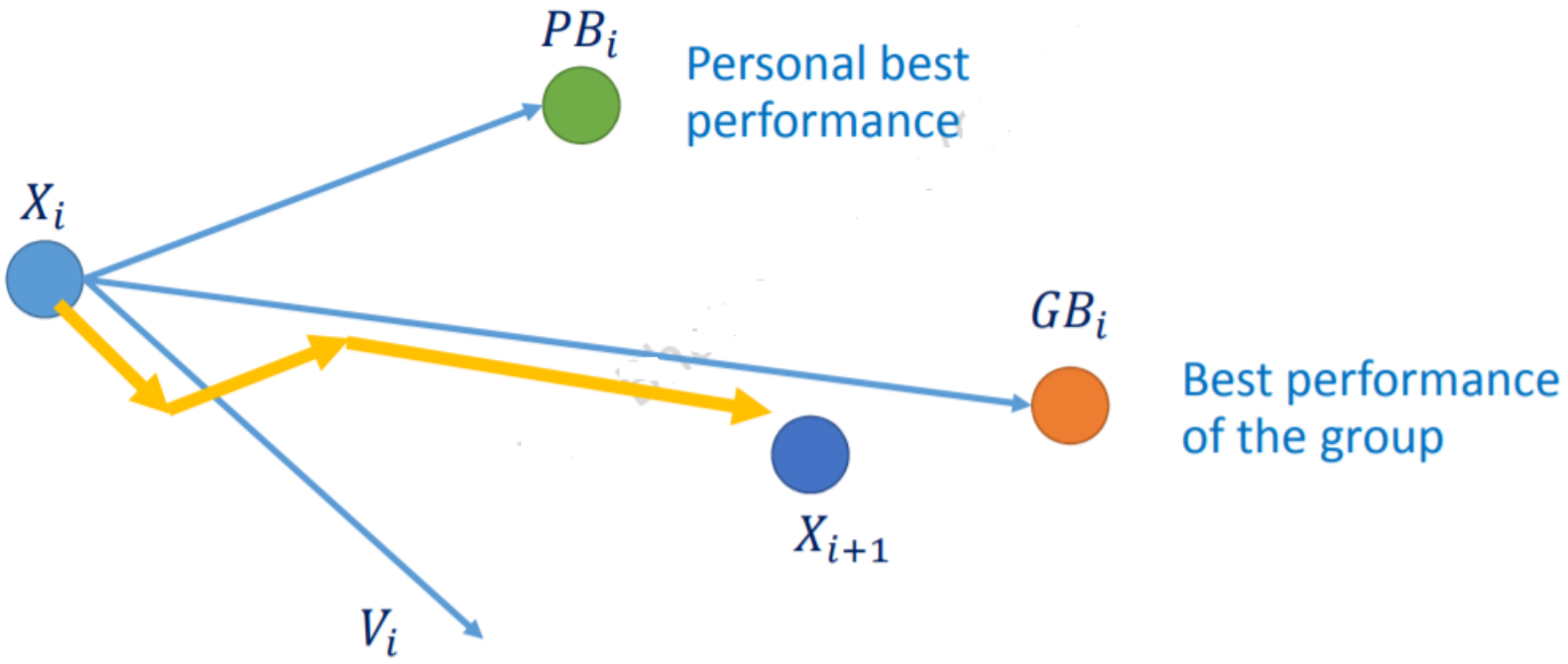
$$V_{i+1} = \omega V_i + C_1 * rand() * (PB_i - X_i) + C_2 * rand() * (GB_i - X_i)$$

Position is updated

$$X_{i+1} = X_i + V_{i+1}$$

C_1 and C_2 are the learning factor
ω is the inertia weight

PSO

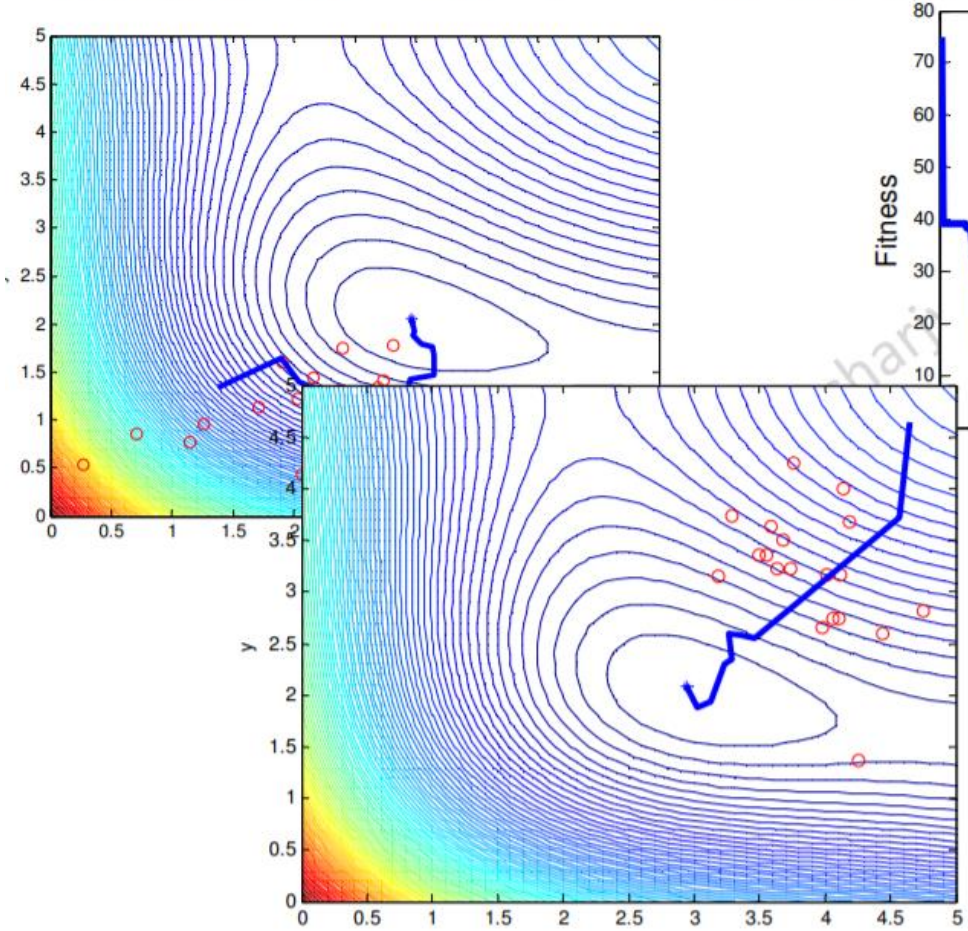

PB_i
Personal best performance
X_i
GB_i
Best performance of the group
X_{i+1}
V_i

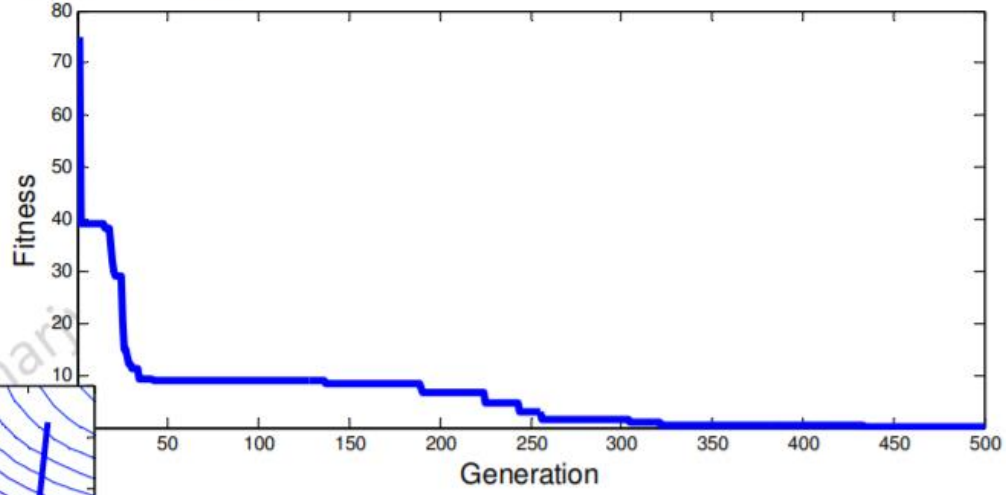
Pseudocode of PSO

```
[x*] = PSO()
P = Particle_Initialization();
For i=1 to it_max
  For each particle p in P do
    fp = f(p);
    If fp is better than f(pBest)
        pBest = p;
    end
  end
  gBest = best p in P;
  For each particle p in P do
      v = v + c1*rand*(pBest – p) + c2*rand*(gBest – p);
      p = p + v;
  end
end
```

Example problem

Minimize $f(x,y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$





Optimum Solution is $x^* = 3, \quad y^* = 2$